

## ▼ Функции в Python.

Функциите представляват част от код, която се изпълнява само когато бъде извикана. Можем да "пращаме" данни на това парче код (чрез параметри на функцията - function parameters) и можем да получаваме данни от същото парче код (посредством ключова дума return).

### Създаване на функция

В Python създаваме функция по следния начин:

```
1 def my_function():  
2     print('Hello world!')
```

Функциите могат да приемат параметри:

```
1 def sum_nums(parameter1, parameter2):  
2     print(parameter1 + parameter2)  
3  
4  
5 sum_nums(5, 10)
```

☞ 15

Както и да връщат стойност:

```
1 def sum_nums(num1, num2):  
2     return num1 + num2  
3  
4  
5 sum = sum_nums(23, 32)  
6  
7 print(sum)  
  
55
```

**В практиката често думите параметър и аргумент се използват взаимозаменяемо. Това, обаче е неточност. Параметър е променливата, която е посочена, че функцията ще приема при дефиницията си, докато аргумент е стойността, която изпращаме при извикване на функцията.**

Функцията трябва да бъде извикана с правилен брой аргументи:

```
1 def func(num1, num2):  
2     return num1 / num2  
3  
4 print(func(5))
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-4-dc3e76902f65> in <cell line: 4>()  
      2     return num1 / num2  
      3  
----> 4 print(func(5))  
  
TypeError: func() missing 1 required positional argument: 'num2'
```

SEARCH STACK OVERFLOW

```
1 def func(num1, num2):  
2     return num1 / num2  
3  
4 print(func(5, 10, 32))
```

```

-----
TypeError                                Traceback (most recent call last)
<ipython-input-5-0f7a4aed816e> in <cell line: 4>()
      2     return num1 / num2
      3
----> 4 print(func(5, 10, 32))

TypeError: func() takes 2 positional arguments but 3 were given

```

## ▼ \*args - Arbitrary arguments

Ако не знаем колко аргумента ще предадем на функция, може да използваме символа \* пред името на параметъра.

```

1 def my_func(*names):
2     for name in names:
3         print(name)
4
5
6 my_func('John', 'Paul', 'Mike', 'Ben', 'Phil')

John
Paul
Mike
Ben
Phil

```

## ▼ Keyword Arguments

Може да изпращаме аргументи със ключ->стойност синтаксиса, познат от речниците.

```

1 def my_func(name1, name2, name3):
2     print('All names: ', name1, name2, name3)
3
4
5 my_func(name3='Phil', name1='John', name2='Paul')

All names:  John Paul Phil

```

## ▼ \*\*kwargs - Arbitrary Keyword Arguments

Ако не е ясно колко keyword аргументи ще бъдат предадени на функцията може да добавим \*\* преди името на параметъра при дефиниране.

```

1 def my_func(**name):
2     print('First name:', name['first_name'])
3     print('Last name:', name['last_name'])
4
5 my_func(last_name='Johnson', first_name='John')

First name: John
Last name: Johnson

```

## ▼ Стойност по подразбиране на параметър.

```

1 def my_func(name, age, is_student=False):
2     print(f'Hello, {name}! You are {age} years old!')
3     if is_student:
4         print('I love learning!')
5
6     print('Function finished!\n')
7
8
9 my_func('George', 22)
10 my_func('Peter', 23, True)
11 my_func(name='John', age=21, is_student=False)

```

```
Hello, George! You are 22 years old!  
Function finished!
```

```
Hello, Peter! You are 23 years old!  
I love learning!  
Function finished!
```

```
Hello, John! You are 21 years old!  
Function finished!
```

## ▼ Type hinting в Python.

Модерното програмиране изисква писане на максимално самодокументиращ код, т.е. код, който максимално описва действието си без изрично то да е описано с коментари. Това може да стане чрез качествено именуване на променливи, логически структуриран код и т.н.. Тъй като Python е loosely typed language нямаме точна декларация на типовете, които очакваме да са изпратените аргументи, както и стойността, която връщаме. За да покажем с какви типове очакваме да работи функцията ни можем да използваме type hinting.

```
1 def hello(age:int) -> str:  
2     return f'Hello there! You\'re {age}!'  
3  
4  
5 hello(15)  
  
    'Hello there! You're 15!'
```

Тук сме показали, че очакваме age да бъде цяло число, а стойността на върнатото от функцията - string.

**Type hints НЕ са включени в изпълнението на код. Те са единствено това - подсказка за типът, който ОЧАКВАМЕ!**

```
1 def hello(age:int) -> str:  
2     return f'Hello there! You\'re {age}!'  
3  
4  
5 hello('apple')  
  
    'Hello there! You're apple!'
```

Тук виждаме, че въпреки очакван тип int за възраст, когато подадем string не се генерира грешка.

## Самостоятелни задачи.

1. Напишете функция, която събира всички елементи в лист.
2. Напишете функция, която изкарва четните числа в даден лист.
3. Напишете функция, която проверява дали даден string е панграм (панграм е изречение, включващо всички букви от дадена азбука).
4. Създайте функция, която изчислява факториел на дадено число. Решете задачата итеративно и рекурсивно.